Small Models that Forget: Exploiting Temporal Locality with Online Learning at Inference Time

Ruben Tous
Affiliation

Draft, October 30, 2026

Abstract

Many inputs that machine learning models see at inference time have strong temporal locality: consecutive inputs tend to be similar and come from a small part of the input distribution. A model facing such inputs does not need to keep everything it knows ready at all times. It may be enough to remember some basic knowledge and adapt quickly to the current context. We study whether this allows smaller models. Multilayer perceptrons of decreasing size are trained offline on MNIST and then updated online, one image at a time, while they classify a stream of test images arranged in bursts of three digits. A shuffled copy of the same stream serves as a control: it checks that the online gains come from temporal locality and not merely from training on more data. With plain SGD, small models trained online classify the bursty stream more accurately than their frozen offline version, and the gain over the shuffled stream grows as the model shrinks and as the online learning rate increases. For the smallest model (6.2 KB), online learning raises accuracy from 64.3% to 70.7%, while on the shuffled stream it stays at 63.7%. Splitting the weights into slow and fast parts helps when the fast part is only the output bias, which raises the smallest model to 76.9%, but a running label prior that learns no weights does better (77.7%). Fast parts on whole layers help much less. In this stream the locality is only in the labels, which favors methods that track the recent labels.

1 Introduction

A trained network is usually deployed frozen: it must classify any input from the whole distribution it was trained on. In many applications, however, the inputs arrive in a stream with strong temporal locality. A camera in a kitchen sees kitchen objects for hours; a speech recognizer hears the same speaker for a whole conversation; a sensor stays in the same regime for long periods. At any moment only a small part of the input distribution is active.

Computer memory hierarchies have relied on this property for decades: a small cache that holds the recently used data serves most requests [2]. We ask whether the same idea applies to the knowledge stored in a network. A model does not need to remember everything all the time if its inputs have strong temporal locality. It may only need some basic knowledge plus the ability to adapt fast to the current context.

Our hypothesis is that, under severe capacity limits, online training at inference time can outperform the same model trained offline and then frozen. A frozen small model must spread its limited capacity over the whole distribution. An online model can spend it on the part of the distribution that is active now. The price is forgetting: neural networks trained sequentially lose what they learned earlier, a behavior known as catastrophic forgetting [11, 3]. This recency bias is usually treated as a problem. In a stream with temporal locality it may be useful, because what was seen recently is what will be seen next.

This paper reports a first set of experiments on MNIST [9]. Experiment 1 checks whether small networks exploit temporal locality when they keep learning online with plain SGD, and how the online learning rate matters. Experiment 2 splits knowledge by time scale, with slow weights and fast weights that adapt to the current context and then fade, and compares it with a running label prior.

2 Related work

Catastrophic forgetting and continual learning. Continual learning tries to learn a sequence of tasks without forgetting the earlier ones [12]. Elastic weight consolidation [7], for example, protects the weights that matter for old tasks. Our setting reverses the goal: we accept forgetting and ask whether it can be used.

Two time scales. Hinton and Plaut [5] added to each connection a fast weight that learns quickly and decays, on top of a slow weight, to recover old memories after interference. Complementary learning systems theory [10] explains the division between a fast-learning hippocampus and a slow-learning neocortex by the need to learn new experiences without overwriting old knowledge. Fast weights in recurrent networks [1] store recent hidden states in a decaying outer-product memory. Experiment 2 uses their formulation but restricts the fast weights to a subset of the layers, and its fast part tracks the current context rather than restoring old associations.

Test time adaptation. Several methods update a model while it is used, for example with a self supervised task [13] or by minimizing the entropy of the predictions [14]. They target distribution shift and do not use labels. Dynamic evaluation [8] updates a language model on the recent part of the sequence it is predicting, which exploits local regularities in text, the closest idea to ours.

Model compression. Compression reduces model size by pruning, quantization or distillation [4], keeping the model frozen. We study a different route to small models that is complementary to compression.

3 Method

3.1 Input streams

The stream contains 10,000 positions built from MNIST test images. It is divided into 193 bursts of random length between 10 and 95 images (51.8 on average). Each burst picks 3 distinct digits at random, and every position in the burst is a random test image of one of those digits, drawn with replacement. The stream holds 6,322 distinct test images. As a reference for how much locality the stream has, predicting the label of the previous image is correct 32.3% of the time.

The control stream is a random permutation of the same positions. It contains exactly the same images, so any difference in accuracy comes from their order. On the control stream, predicting the previous label is correct 9.9% of the time. The control is not a scenario we expect in practice: we assume that the real input streams have temporal locality and are not random. Its only purpose is to show whether a gain from online learning comes from that locality or merely from training on more data.

The images of a burst are drawn independently given their digit, so the locality of the stream is in the labels only.

3.2 Protocol

Each model is first trained offline on the 60,000 MNIST training images. We call this the frozen model. Then it processes a stream one image at a time. For each image, the model first predicts its label, and the prediction counts towards the online accuracy. Then it takes one optimization step on that single image with its true label. This assumes that the label becomes available after each prediction, as in prequential evaluation. Each stream starts from the same frozen weights.

We report these quantities, all on the same 10,000 positions:

- *frozen accuracy*: the frozen model without updates (equal on both streams, since they hold the same images);
- *online accuracy on bursts* and *online accuracy on random*: the online model on each stream;
- *online gain*: online accuracy on bursts minus frozen accuracy;
- *locality effect*: online accuracy on bursts minus online accuracy on random.

Differences are given in percentage points (pp). The online gain tells whether online learning is worth it on the bursty stream. The locality effect isolates the part of the gain that comes from temporal locality. It removes the effect of simply training more, and of seeing some test images more than once, which happens in both streams. Online accuracy on random minus frozen accuracy shows what online learning would give without locality, that is, the effect of the extra training alone.

Model size is the memory needed to store all parameters and buffers as 32 bit floats. All runs use a fixed seed (0).

3.3 Models

The multilayer perceptron (MLP) has one hidden layer with ReLU: $784 \to h \to 10$, with h between 128 and 2. It is trained offline with Adam [6], learning rate 0.001, batch size 128, 10 epochs. The online phase uses plain SGD without momentum.

4 Experiments

4.1 Experiment 1: online learning rate

The first experiment updates MLPs with $h \in \{128, 16, 8, 4, 2\}$ online with SGD at four learning rates: 0.0003, 0.001, 0.003 and 0.01. Table 1 gives the online gain and the locality effect, and Table 2 the change on the shuffled stream. Figures 1 and 2 show the results against model size.

With SGD and learning rates up to 0.001, online learning improves on the frozen model at every size. The online gain grows as the model shrinks: 0.2 to 0.4 pp for $h \geq 16$, and 6.44 pp for $h = 2$ at learning rate 0.001 (from 64.27% to 70.71%). On the shuffled stream the same small model stays near where it was (63.72%), so the gain comes from the order of the inputs and not from the extra training. Learning rate 0.001 gives the largest online gain for $h = 16$, 8 and 4; for $h = 2$, 0.003 is level with it (+6.45 against +6.44 pp), and for $h = 128$, 0.0003 is ahead (+0.33 against +0.19 pp).

The locality effect grows in two directions: as the model gets smaller and as the learning rate gets larger. At learning rate 0.01 it reaches 12.58 pp for $h = 4$, but there the online model is worse than the frozen one on both streams: the effect is large because accuracy on the shuffled stream collapses faster (Table 2). Between these extremes, learning rate 0.003 matches 0.001 on bursts for $h = 2$ but is 4.5 pp below the frozen model on the shuffled control, where 0.001 is 0.6 below: a faster rate relies more on the recent past, and so more of its gain is due to locality. The large model

Table 1: Experiment 1: MLPs with online SGD at four learning rates. Online gain (bursts minus frozen) and locality effect (bursts minus random), in pp.

			online gain				locality effect			
h	KB	frozen	0.0003	0.001	0.003	0.01	0.0003	0.001	0.003	0.01
128	397.5	97.86	+0.33	+0.19	−0.22	−2.62	+0.16	−0.05	−0.15	+0.87
16	49.7	95.04	+0.25	+0.37	+0.25	−4.22	+0.05	+0.19	+1.38	+2.49
8	24.9	92.59	+0.75	+1.36	+0.72	−2.88	+0.48	+1.49	+2.35	+4.30
4	12.5	85.85	+1.38	+2.30	+1.59	−4.28	+1.49	+3.69	+5.55	+12.58
2	6.2	64.27	+4.25	+6.44	+6.45	−4.50	+2.98	+6.99	+10.93	+11.70

Table 2: Experiment 1: change on the shuffled stream (online accuracy on random minus frozen), in pp.

h	0.0003	0.001	0.003	0.01
128	+0.17	+0.24	−0.07	−3.49
16	+0.20	+0.18	−1.13	−6.71
8	+0.27	−0.13	−1.63	−7.18
4	−0.11	−1.39	−3.96	−16.86
2	+1.27	−0.55	−4.48	−16.20

($h = 128$) shows almost no locality effect at learning rates up to 0.003. With enough capacity, the frozen model already covers the whole distribution and there is little to gain from adapting to the current context.

4.2 Experiment 2: two time scales

The online loss only sees the current image, so plain online SGD has to trade adapting to the current burst against keeping what it knows. The second experiment splits knowledge by time scale [5]. Each chosen parameter gets a slow part S and a fast part F, and the network uses $S + F$. After each image, with g the gradient of the loss on that image,

$$S \leftarrow S - \eta_{\text{slow}}\, g,$$
$$F \leftarrow \lambda F - \eta_{\text{fast}}\, g,$$

with F starting at zero. The fast part is a decaying sum of recent gradients: it follows the current burst and forgets it within about $1/(1-\lambda)$ images.

We compare six online variants, on MLPs with $h \in \{128, 8, 4, 2\}$. The four fast variants vary which connections carry a fast weight, from the output bias alone to every connection.

- *sgd*: plain SGD, $\eta_{\text{slow}} = 0.001$, no fast part (the best setting of Experiment 1);
- *fast bias*: $\eta_{\text{slow}} = 0.001$, with a fast part on the output bias only ($\eta_{\text{fast}} = 1$, $\lambda = 0.97$), which adds 10 values;
- *fast out*: $\eta_{\text{slow}} = 0.001$, with a fast part on the output layer, weights and bias ($\eta_{\text{fast}} = 0.003$, $\lambda = 0.97$), which adds $10h + 10$ values;

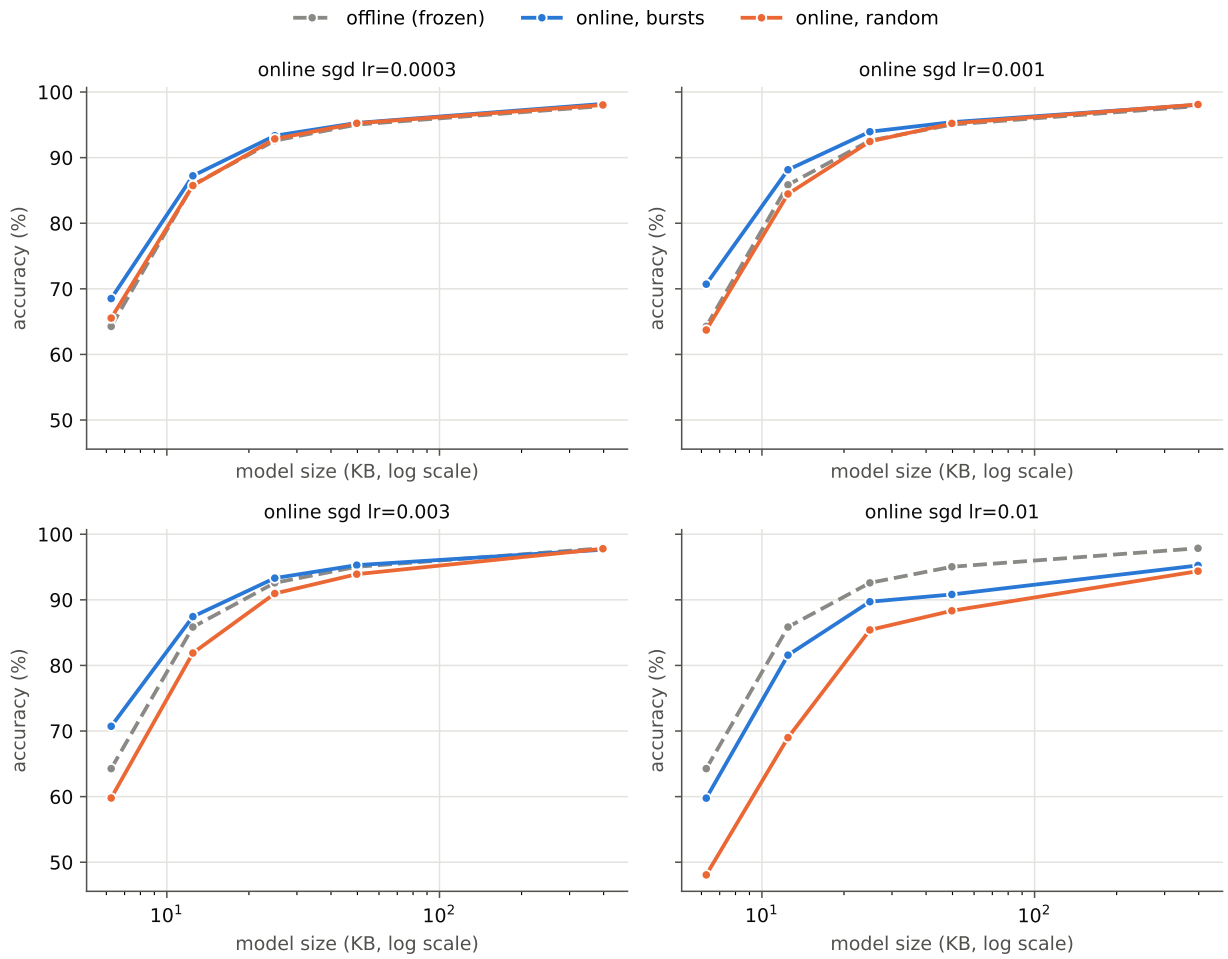


Figure 1: Experiment 1: accuracy against model size, one panel per online learning rate.

- *fast hidden*: $\eta_{\text{slow}} = 0.001$, with a fast part on the first layer, weights and bias ($\eta_{\text{fast}} = 0.003$, $\lambda = 0.97$), which adds $785h$ values;
- *fast all*: $\eta_{\text{slow}} = 0.001$, with a fast part on every weight and bias ($\eta_{\text{fast}} = 0.003$, $\lambda = 0.99$), which doubles the model;
- *label prior*: frozen weights and a running estimate p of the label frequencies, $p \leftarrow 0.9\,p + 0.1\,\text{onehot}(y)$ after each prediction, starting uniform and kept $\geq 10^{-3}$. Prediction adds $\log p$ to the logits, so digits absent from the recent past become very unlikely. It learns no weights and adds 10 values.

The label prior is a baseline that exploits exactly the locality in the labels and nothing else. The fast-part hyperparameters were tuned on the same stream they are scored on. For fast out and fast hidden we searched $\eta_{\text{fast}} \in \{0.003, 0.01, 0.03, 0.1, 0.3, 1\}$ and $\lambda \in \{0.97, 0.99\}$ and kept the setting with the best mean online gain over the four sizes. Both chose $\eta_{\text{fast}} = 0.003$, the smallest value in the grid; the gain fell steadily as η_{fast} grew. As a ceiling, we also score an oracle: the frozen model with its prediction restricted to the digits of the current burst, which it knows in advance.

Table 3 gives the online gain, Table 4 the locality effect and the change on the shuffled control, and Figure 3 shows the online gain and the locality effect against model size.

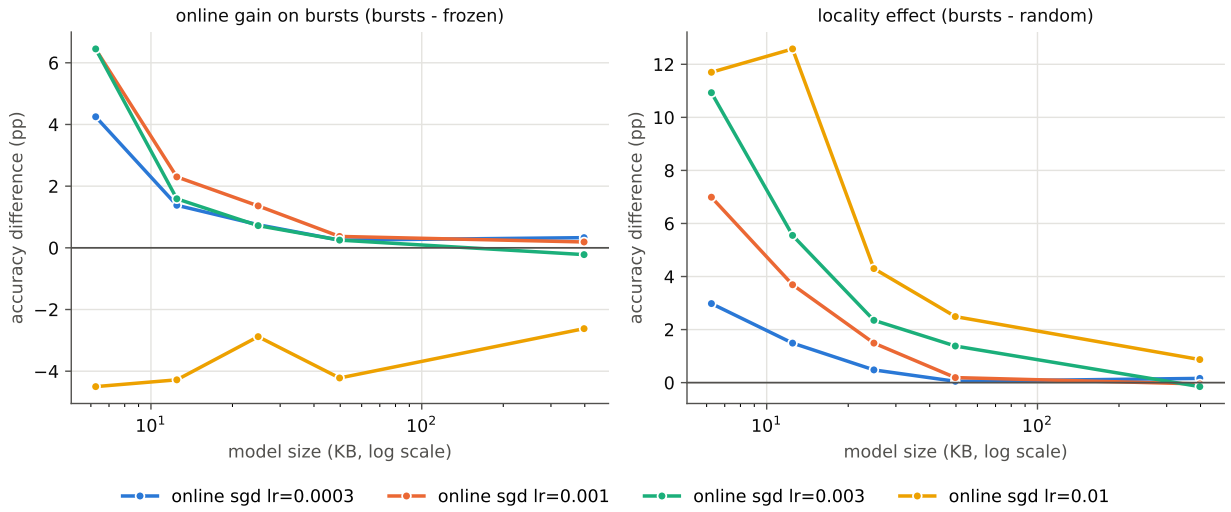


Figure 2: Experiment 1: online gain (left) and locality effect (right) against model size, one line per online learning rate.

Table 3: Experiment 2: online gain on the bursty stream (bursts minus frozen) for the six variants, in pp. The oracle restricts the frozen model to the digits of the current burst.

h	frozen	sgd	prior	f. bias	f. out	f. hidden	f. all	oracle
128	97.86	+0.19	+0.80	+0.30	+0.39	+0.27	+0.17	+1.45
8	92.59	+1.36	+2.76	+1.83	+1.51	+1.68	+1.34	+4.83
4	85.85	+2.30	+5.17	+3.51	+2.48	+2.50	+2.39	+8.86
2	64.27	+6.44	+13.43	+12.66	+8.34	+6.49	+7.92	+21.48

The label prior gains most at every size (+13.43 pp for $h = 2$, 63% of the oracle's +21.48), without learning any weights. A fast output bias is second at every size and comes close to the prior for $h = 2$ (+12.66 pp), nearly double plain SGD (+6.44 pp). Fast parts on whole layers gain much less. A fast output layer gains +8.34 pp for $h = 2$, about a third of the way from plain SGD to the fast bias, and is level with plain SGD for $h = 4$ and $h = 8$. Its fast learning rate had to be over 300 times smaller than the bias's, because larger rates on the output weights hurt at every size. A fast first layer is level with plain SGD for $h = 2$ (+6.49 pp) and slightly above it for $h = 8$ and $h = 4$. Making the whole network fast is level with the fast output layer (+7.92 pp for $h = 2$), and it doubles the model.

On the shuffled control every variant ends up at or below the frozen model for $h \leq 8$ (for $h = 2$: label prior −10.57 pp, fast variants −1.19 to −7.32 pp, plain SGD −0.55 pp), so their gains on the bursty stream come from its temporal locality and not from the extra training. The prior depends on locality most, since it nearly rules out the digits it has not seen recently.

5 Discussion

The experiments support the hypothesis for small MLPs. When capacity is very limited, a model that keeps adapting to the current context classifies a stream with temporal locality better than the same model frozen, and the advantage comes from the locality: it disappears or reverses on a shuffled copy of the stream. The recency bias of online training, which hurts on the shuffled

Table 4: Experiment 2: locality effect (bursts minus random) and change on the shuffled control (random minus frozen) for the six variants, in pp.

	locality effect						shuffled control					
h	sgd	prior	bias	out	hid.	all	sgd	prior	bias	out	hid.	all
128	−0.05	+1.22	+0.09	+0.17	+0.04	+0.34	+0.24	−0.42	+0.21	+0.22	+0.23	−0.17
8	+1.49	+4.78	+2.46	+2.82	+2.09	+3.63	−0.13	−2.02	−0.63	−1.31	−0.41	−2.29
4	+3.69	+9.83	+6.31	+6.11	+4.53	+7.77	−1.39	−4.66	−2.80	−3.63	−2.03	−5.38
2	+6.99	+24.00	+19.98	+13.05	+7.68	+14.38	−0.55	−10.57	−7.32	−4.71	−1.19	−6.46

bias, out, hid., all: fast bias, fast out, fast hidden, fast all.

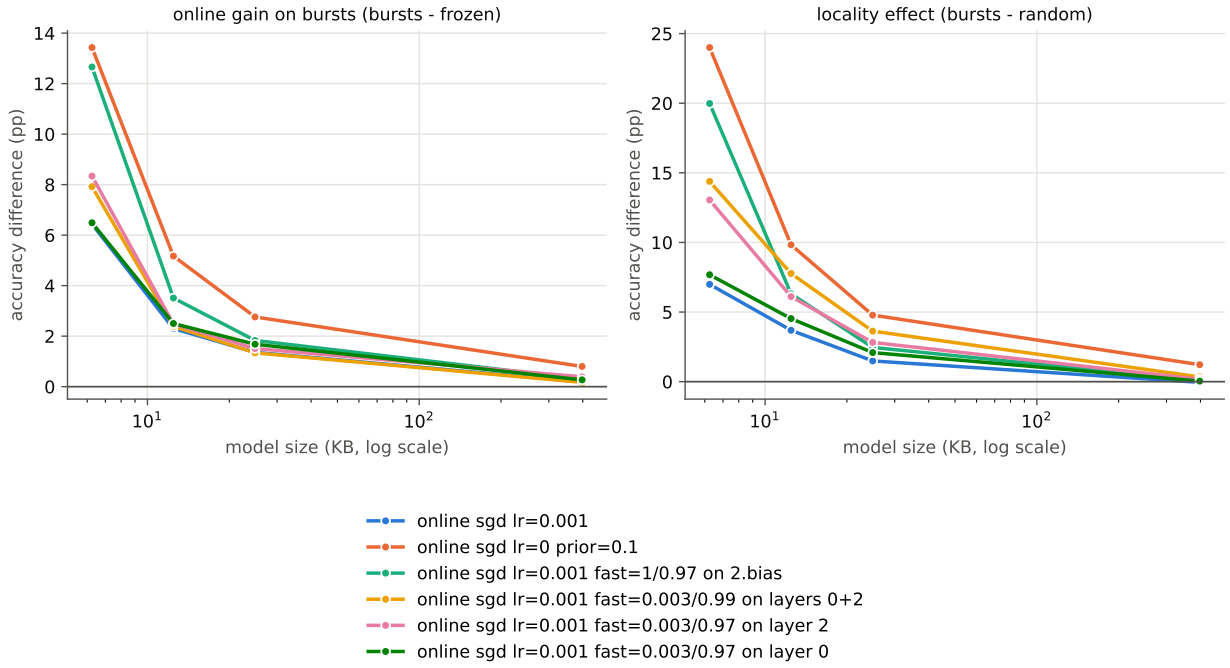


Figure 3: Experiment 2: online gain (left) and locality effect (right) against model size, one line per variant.

control, helps on the bursty one. Seen this way, the learning rate controls the length of the model's memory: a larger rate forgets faster and adapts faster. The best value should depend on how long the contexts last, which here is about 50 images.

The shuffled control is not a deployment scenario: we assume that real streams have temporal locality. It shows that every way of exploiting the burst more strongly, from a faster learning rate to fast weights or a label prior, relies more on that locality, and would not help on a stream without it.

The strength of the label prior says something about the stream rather than about fast weights. The images of a burst share only their labels, so the most useful thing to remember about the recent past is which digits are active. A fast output bias can learn this with a large fast learning rate, and that is what it does best. Fast parts on weights need a much smaller rate, and they gain less whether they sit in the output layer, the first layer or both. A stream whose locality is also in the inputs would be a fairer test of fast weights, because a label prior cannot exploit it. A fast first

layer is the natural variant to test there, since a shared input distortion would have to be absorbed in the first layer.

The results also have clear limits. The task is MNIST and the stream is synthetic, with an idealized form of locality: bursts with sharp boundaries and only three classes each. The online model receives the true label right after each prediction, which many applications cannot provide. Every result comes from a single run with seed 0, so differences below about 0.5 pp may not be meaningful. The fast-weight hyperparameters of Experiment 2 were tuned on the same stream they are scored on. The stream draws test images with replacement, so the online model sees some images more than once; the shuffled control has the same repetitions, and the locality effect cancels this out, but the online gain does not.

6 Conclusion and next steps

Small MLPs trained online at inference time exploit the temporal locality of their input stream. Below some model size they become more accurate than their frozen offline version, and the advantage is tied to the order of the inputs. The effect grows as the model shrinks and as the online learning rate increases, and the shuffled control confirms that it comes from temporal locality and not from training on more data. A fast output bias on top of slow weights nearly doubles the gain of plain SGD for the smallest model, but a running label prior without any learning does better, because the locality of the stream is in the labels. Fast parts on whole layers gain much less than the fast bias.

Next steps include streams with locality in the inputs (for example one shared distortion per burst), repeating the runs with several seeds, tuning on separate streams, varying the burst length and the number of classes per burst, testing streams with natural locality (for example video) and other architectures, and replacing the true labels with weaker feedback.

References

[1] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems (NIPS)*, 2016.

[2] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[3] Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.

[4] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] Geoffrey E. Hinton and David C. Plaut. Using fast weights to deblur old memories. In *Proceedings of the 9th Annual Conference of the Cognitive Science Society*, pages 177–186, 1987.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[7] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[8] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[9] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[10] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[11] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[12] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

[13] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[14] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*, 2021.